Modularisation par les Rôles et Sécurisation avec Ansible Vault

1. Contexte opérationnel

L'infrastructure sous Debian est désormais déployée et sous contrôle d'Ansible.

- **Nœud de contrôle (Master) :** `192.168.4.100` (compte : `formation`)
- **Nœud géré 1 (node1) :** `192.168.4.101`
- **Nœud géré 2 (node2) :** `192.168.4.102`

L'équipe d'exploitation constate que la gestion via un playbook monolithique unique devient complexe à maintenir. De plus, des données sensibles sont actuellement stockées en clair dans les fichiers d'automatisation.

La mission consiste à refondre l'Infrastructure as Code (IaC) sous la forme d'un projet modulaire basé sur des rôles Ansible standardisés, en y intégrant la gestion des secrets avec Ansible Vault.

Consigne opérationnelle : Toute l'administration s'effectue exclusivement depuis le Master dans le dossier `~/tp-ansible-roles/`. Les connexions manuelles interactives sur les nœuds cibles sont proscrites.

2. Objectifs techniques

- Initialiser une arborescence de rôles respectant les conventions Ansible Galaxy.
- Découper la configuration en briques unitaires : un rôle de socle technique (`common_baseline`) et un rôle applicatif (`web_nginx`).
- Sécuriser les identifiants critiques à l'aide d'Ansible Vault.
- Automatiser l'authentification Vault sans saisie interactive lors de l'exécution.
- Valider la conformité du parc et l'idempotence stricte des tâches.

3. Cahier des charges

Étape 1 : Initialisation de l'environnement

Dans le répertoire `~/tp-ansible-roles/` sur le Master :

1. Déclarez le mot de passe maître dans le fichier local caché `~/.vault_pass` avec des permissions strictes (`0400`). Le secret maître retenu est : `VaultKeyMaster#2026`.

2. Créez un fichier `ansible.cfg` paramétré pour :
    - Pointer sur l'inventaire local `./inventory.ini`.
    - Désactiver le contrôle strict de clé d'hôte (`host_key_checking = False`).
    - Référencer automatiquement le fichier de mot de passe Vault (`vault_password_file = ~/.vault_pass`).
3. Configurez `inventory.ini` avec le groupe `production_nodes` contenant `node1` et `node2`. Définissez les variables d'accès (`ansible_user=formation`, interpréteur Python).

Étape 2 : Conception du rôle de socle (`common_baseline`)

Générez la structure du rôle dans `roles/common_baseline` avec `ansible-galaxy role init`.

Ce rôle doit assurer les opérations suivantes :

1. **Socle logiciel :**
    - Mettre à jour le cache APT (si plus vieux que 3600 secondes).
    - Installer les utilitaires d'exploitation : `curl` et `htop`.
2. **Compte d'audit :**
    - S'assurer de la présence du compte système `sys-auditor` avec son répertoire personnel et un shell Bash.
    - Injecter la clé publique (`~/.ssh/id_auditor.pub`) générée sur le Master via le module `authorized_key` et la fonction `lookup`.

Étape 3 : Conception du rôle applicatif (`web_nginx`)

Générez la structure du rôle dans `roles/web_nginx` avec `ansible-galaxy role init`.

Ce rôle doit encapsuler le cycle de vie de Nginx :

1. **Variables par défaut (`defaults/main.yml`) :**
    - `web_nginx_port: 80`
    - `web_nginx_root_dir: "/var/www/html"`
2. **Tâches (`tasks/main.yml`) :**
    - Installer le paquet `nginx`.
    - Déployer un gabarit de virtualhost personnalisé (`templates/site.conf.j2`) dans `/etc/nginx/sites-available/custom.conf`.
    - Activer le site via un lien symbolique dans `/etc/nginx/sites-enabled/custom.conf`.

- Supprimer le site `default` natif dans `/etc/nginx/sites-enabled/default`.
- Déployer une page dynamique `templates/index.html.j2` dans `{{ web_nginx_root_dir }}/index.html`.

3. **Gestionnaires (`handlers/main.yml`) :**
    - Prévoir un handler nommé `Recharger Nginx` qui recharge le service, notifié lors des modifications de configuration.
4. **Intégration dynamique Jinja2 :**
    - La configuration Nginx doit écouter sur la variable `{{ web_nginx_port }}`.
    - La page web HTML générée doit afficher le nom de la machine (`{{ ansible_hostname }}`), son IP et le port d'écoute actif.

Étape 4 : Gestion des secrets avec Vault

1. Créez un dossier `vars/` à la racine de votre projet.
2. Créez et chiffrez le fichier `vars/vault.yml` avec la commande `ansible-vault` (protégé par le mot de passe défini à l'étape 1).
3. Ce fichier chiffré doit contenir la variable suivante : `vault_secret_token: "TOKEN-PROD-9982"`
4. Modifiez la page d'accueil web (`index.html.j2` du rôle Nginx) pour y afficher la variable `{{ vault_secret_token }}`.

Étape 5 : Orchestration et surcharge de variables

Rédigez le playbook maître `site.yml` à la racine du projet :

1. Ciblez le groupe `production_nodes` avec élévation de privilèges (`become: true`).
2. Chargez les données sensibles chiffrées via la directive `vars_files` pointant sur `vars/vault.yml`.
3. Orchestrez les appels de rôles :
    - Appliquer `common_baseline` sur l'ensemble des nœuds.
    - Appliquer `web_nginx` avec les valeurs par défaut sur `node1`.
    - Appliquer `web_nginx` sur `node2` en surchargeant la variable `web_nginx_port` à la valeur `8080`.

4. Critères

Pour valider l'atelier, exécutez les contrôles suivants depuis le Master :

1. **Exécution automatisée :** Le playbook s'exécute de bout en bout sans invite de mot de passe interactif (`ansible-playbook site.yml`).
2. **Preuve formelle d'idempotence :** Une réexécution immédiate de la commande aboutit à un état neutre (`changed=0` sur l'ensemble des cibles).
3. **Validation fonctionnelle des ports et du chiffrement :** Les requêtes HTTP suivantes doivent aboutir et afficher le contenu de la page web incluant le token secret déchiffré.
    - `curl -s http://192.168.4.101:80` (réussite attendue)
    - `curl -s http://192.168.4.102:8080` (réussite attendue)
    - `curl -s http://192.168.4.102:80` (échec attendu sur ce port)